

# NUT AI

## A Designed, Not Cloned, Mobile Experience

A pattern-grounded blueprint for transforming the Nut AI mobile app: a shadcn-grade token system, disciplined iconography, rebuilt navigation, motion and haptics, screen-by-screen redesigns, and a bloat census - researched across the shadcn ecosystem and Mobbin-class pattern libraries. App code untouched; this is the design of the redesign.

### Nut AI Engineering

Companion volume to the Full QA Audit Report · Expo / React Native

# Table of Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. Executive Summary</b>                              | <b>3</b>  |
| <b>2. Research Methodology and Source Map</b>            | <b>4</b>  |
| <b>3. Current State Audit: Where the UI Stands Today</b> | <b>5</b>  |
| 3.1 Screen census and navigation                         | 5         |
| 3.2 Theme tokens: a real foundation with holes           | 6         |
| 3.3 Typography: seventeen sizes and no font              | 6         |
| 3.4 Icons: a differentiated asset, barely used           | 7         |
| 3.5 Components: five chip families and four Fields       | 8         |
| 3.6 States, interaction and accessibility                | 8         |
| <b>4. Design Direction: Compose, Do Not Clone</b>        | <b>9</b>  |
| 4.1 Three theme directions                               | 9         |
| 4.2 Typography decision                                  | 10        |
| 4.3 Spacing, radius, elevation and state layers          | 11        |
| <b>5. The shadcn Way on Mobile</b>                       | <b>11</b> |
| 5.1 The React Native reality: react-native-reusables     | 11        |
| 5.2 Two adoption strategies, one decision                | 11        |
| 5.3 Migration order                                      | 12        |
| <b>6. Iconography System</b>                             | <b>13</b> |
| 6.1 Rules borrowed from Apple                            | 13        |
| 6.2 Keep custom, align to Lucide, fill gaps              | 13        |
| 6.3 Deployment targets                                   | 14        |
| <b>7. Navigation and Information Architecture</b>        | <b>15</b> |
| 7.1 Fixing the Home / Food duplication                   | 15        |
| 7.2 Header unification and modal hygiene                 | 16        |
| <b>8. Screen-by-Screen Transformation Blueprints</b>     | <b>16</b> |
| 8.1 Onboarding: 22 screens to 12                         | 16        |
| 8.2 Home: an instrument, not a dashboard                 | 17        |
| 8.3 Food logging: from forms to flow                     | 17        |
| 8.4 Scan and result: confidence you can see              | 17        |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| 8.5 Training: the Hevy pattern . . . . .                           | 18        |
| 8.6 Progress and reports . . . . .                                 | 18        |
| 8.7 Assistant: a conversation that feels engineered . . . . .      | 18        |
| 8.8 Profile and settings: progressive disclosure . . . . .         | 18        |
| <b>9. Interaction, Motion and Haptics System . . . . .</b>         | <b>19</b> |
| 9.1 Haptics map . . . . .                                          | 19        |
| 9.2 Skeletons: the NN/g decision rules . . . . .                   | 20        |
| <b>10. States and Feedback Overhaul . . . . .</b>                  | <b>21</b> |
| 10.1 One toast system, two rules . . . . .                         | 21        |
| 10.2 Optimistic logging and honest uncertainty . . . . .           | 21        |
| <b>11. Accessibility and Visual Trust . . . . .</b>                | <b>22</b> |
| 11.1 Dynamic type and VoiceOver plan . . . . .                     | 22        |
| <b>12. Bloat Removal Census . . . . .</b>                          | <b>23</b> |
| <b>13. Implementation Roadmap . . . . .</b>                        | <b>25</b> |
| <b>Appendix . . . . .</b>                                          | <b>26</b> |
| A Source bibliography . . . . .                                    | 26        |
| B Current token dump (verbatim, for migration reference) . . . . . | 26        |
| C Screen inventory and transformation tag . . . . .                | 27        |

# 1. Executive Summary

Nut AI is functionally one of the deepest offline nutrition engines in its class: 362 curated Indian dishes, a 528-row IFCT and 7,928-food USDA corpus, a six-signal food resolver, adaptive targets and an AI assistant with five tools. Its interface, however, does not yet tell that story. A read-only audit of every screen file found a product that behaves like a well-engineered prototype: a real token system that 15 files bypass with hardcoded colors, a motion spec that nothing animates, a hand-drawn 53-glyph icon set used in only 59 places, and five different ways to draw a chip. The gap between what the app knows and what the app feels like is the single biggest product opportunity left, and it is a design gap, not an engineering one.

|                           |                                     |                                 |                                 |
|---------------------------|-------------------------------------|---------------------------------|---------------------------------|
| 52<br>screens + 3 layouts | 53<br>custom glyph icons, 59 usages | 0<br>custom fonts, 0 animations | 15<br>files bypass color tokens |
|---------------------------|-------------------------------------|---------------------------------|---------------------------------|

This report is the design of that redesign. It deliberately changes no code. Instead it does three things. First, it establishes ground truth: an exact inventory of the current theme tokens, typography, iconography, components, states and interaction patterns, with every number traceable to a file. Second, it grounds the transformation in external evidence: the shadcn/ui design philosophy and its React Native port react-native-reusables, Apple Human Interface Guidelines on icons and haptics, Nielsen Norman Group research on skeleton screens, Material Design 3 token architecture, and pattern research across the Mobbin-class source map defined in Chapter 2. Third, it converts all of it into decisions: three theme directions with full token tables, an icon system with a 20-action mapping, navigation and screen-by-screen blueprints, a motion and haptics specification, a bloat census with kill, merge and hide lists, and a four-wave roadmap.

The headline recommendation is to adopt the shadcn way rather than the shadcn stack blindly. shadcn/ui is not a component library; it is a discipline of owning component code, composing from one token system, and shipping beautiful defaults. The React Native port (react-native-reusables, 8.7k stars, built on Nativewind) is excellent but importing it wholesale would mean rewriting 52 screens that currently use StyleSheet-based tokens. The pragmatic path, argued in Chapter 5, is a pattern port: keep the token engine, rebuild the roughly 30 primitives and screens that matter, and adopt library pieces (bottom sheets, toasts, skeletons) where the ecosystem is simply better. The result should feel like the apps users already love: the navigation clarity of Spotify, the information density of Linear, the spacing confidence of Apple, the activity visualization of Strava, and a visual identity that is distinctly Nut AI.

**The ten highest-leverage moves**

1. Re-found the design system on one complete token set (color, type, space, radius, motion, elevation) and route every screen through it. 2. Collapse the type scale from 17 ad-hoc sizes to 7 tokens and fix the 11px accessibility failures. 3. Unify one Screen header with an iOS-style large-title pattern to replace four header variants. 4. Turn the FAB tile grid into a real bottom sheet and merge the duplicated Home and Food timelines. 5. Discipline the icon set: one grid, one stroke, semantic sizes, and lucide fallbacks for missing metaphors. 6. Ship a toast system to replace 36 Alert.alert call sites and two competing web dialog shims. 7. Add skeleton loading, empty states with actions, and pull-to-refresh so no screen is ever a blank spinner. 8. Wire motion and haptics app-wide (reanimated is already installed and unused) with a reduce-motion escape hatch. 9. Rebuild the training screens around Hevy-style row actions instead of a twelve-button wall of equal-weight text. 10. Cut the bloat: dead controls, debug model identifiers in user UI, five near-synonymous save actions, and five low-information onboarding screens.

Nothing in this plan requires a rewrite. Every chapter is sequenced so that the cheapest, highest-trust changes land first, and Chapter 13 gives effort estimates and a definition of done. If only one chapter is acted on, make it Chapter 3 (know exactly what exists today) and Chapter 12 (delete what should not exist). Design systems get better by subtraction before they get better by addition.

## 2. Research Methodology and Source Map

The research followed a pattern-first workflow instead of a genre-first one. Searching for "best fitness app UI" returns portfolio art; searching for the individual problems the app actually has, such as workout history presentation, goal selection, calorie ring dashboards, food logging review, and bottom navigation, returns patterns that can be composed rather than copied. Every screen blueprint in Chapter 8 names the specific problem it solves and the reference pattern it draws from, so each decision can be traced back to a shipped interface rather than a designer fantasy.

The source map below classifies the research corpus by what each class of site is actually good for. Screenshot libraries of shipped apps (Mobbin, Uiguana, Kombai, Banani) are evidence of UX that survived contact with users. Flow libraries (Page Flows) show what happens after the tap. SaaS structure references (SaaSFrame) informed the settings and data-dense screens. Visual trend sources (Dribbble, Godly, Awwwards) were used for aesthetic language only, never for interaction logic, because concept art pretending to be software is the fastest way to import unshippable ideas. Implementation references (Emerald UI, the Awesome UI Libraries index, shadcn.io registries) bridge design to code. Authoritative guidelines (Apple HIG, Material Design 3, Nielsen Norman Group) set the non-negotiable constraints, and shadcn.com plus the bundled ui-ux-pro-max design-intelligence dataset anchored the component and token standards.

**Table 2.1 Source classes and what each contributed**

| Source class             | Representative sources                      | Used for                                            | Weight              |
|--------------------------|---------------------------------------------|-----------------------------------------------------|---------------------|
| Shipped-app libraries    | Mobbin, UIguana, Kombai, Banani             | Real flows: onboarding, logging, paywalls, settings | Primary UX evidence |
| Flow libraries           | Page Flows                                  | What happens after the tap; multi-step flows        | Interaction truth   |
| SaaS structure           | SaaSFrame, saasui.design                    | Dense data screens, tables, settings                | Structure reference |
| Visual language          | Dribbble, Godly, Awwwards                   | Palettes, card styles, type; never flows            | Aesthetic only      |
| Implementation refs      | Emerald UI, Awesome UI Libraries, shadcn.io | How good components are actually built              | Code bridge         |
| Component standard       | shadcn.com docs, ui-ux-pro-max dataset      | Component inventory, states, tokens                 | Design-system spine |
| Authoritative guidelines | Apple HIG, Material 3, NN/g                 | Icons, haptics, loading, a11y, tokens               | Constraints         |
| App reviews              | Hevy, MacroFactor, Cal AI coverage          | What users praise or punish in this genre           | Genre calibration   |

Execution was concrete: 24 targeted web searches across the problem space, 13 full-text page reads, and three local design-intelligence datasets (iconography, typography pairings, product palettes, UX guidelines). The authoritative reads included the shadcn/ui documentation and its principles, the react-native-reusables repository, Apple HIG pages for icons and haptics, the Nielsen Norman Group skeleton-screen research, Material Design 3 token architecture, and current 2026 trend surveys from four independent studios. Where sources disagreed, the hierarchy was: platform guidelines first, shipped app evidence second, trend aesthetics last. The full bibliography appears in Appendix A.

**Why no stock screenshots appear in this report**

Every visual in this document is generated from the app’s actual measured data (usage counts, duplication census, token values). Referencing real product screenshots inside a design spec invites pixel-copying; naming the pattern and its source keeps the borrowing structural. Steal the language, not the layout.

### 3. Current State Audit: Where the UI Stands Today

This chapter is the ground truth every later chapter builds on. All figures come from a read-only inspection of the app source in September 2026, and supersede the earlier QA audit where they disagree: the app now has 52 screen files plus 3 layouts (not 44), 15 files bypass the color tokens (not 8), and onboarding spans 22 files driven by a 20-step flow definition. The audit covers structure, theme, typography, icons, components, states, interaction and accessibility.

#### 3.1 Screen census and navigation

The route tree decomposes into six flows: 22 onboarding files on a shared `Chrome/OptionScreen/Controls` foundation, 9 food-logging screens (including the 1,130-line food-search and the 694-line dish-composer), 8 training screens, 5 tabs, the 686-line streaming assistant, and the scan trio (camera, the 1,091-line result screen, and the 519-line adaptive check-in). Navigation is expo-router with a custom pill tab bar (home, food, train, progress, profile) and a 58x58 FAB visible only on the food tab, which opens a fullscreen modal action grid of six tiles. Twenty screens register as modals; five others, including the assistant, push as cards, so the two full-screen "focus" experiences animate differently for no functional reason. The deep-link scheme `nutai` is declared in `app.config.ts` and promised in the README, but no handler exists anywhere in code.

## 3.2 Theme tokens: a real foundation with holes

`src/theme/tokens.ts` is a genuinely good start: an 11-step ink neutral ramp, three macro identity colors (protein blue `#3E7BFA`, carbs amber `#F2A93B`, fat violet `#7B5EA7`), a full light and dark semantic theme, a 4pt spacing scale, six radii, an eight-step type scale, five motion durations, and a lint-enforced 44pt minimum tap target. Three holes matter. There are no elevation or shadow tokens, so depth is improvised twice in the whole app. Dark mode ships with a code comment reading "designed to AA targets but have never been computed", and this report confirms several failures in Chapter 11. And `ThemeProvider` is context-only: there is no accent-color concept, no theme persistence, and no way for a screen to consume a semantic alias that does not already exist.

## 3.3 Typography: seventeen sizes and no font

The app loads no fonts at all; everything renders in the platform system face with numeric weights 400 to 800. That is a defensible choice (system fonts are fast and native-feeling), but it makes the type scale the only lever, and the scale is bypassed roughly 60 percent of the time. Across app and shared components there are 17 distinct ad-hoc `fontSize` values (12, 15, 17, 18, 22, 23, 24, 26, 30, 32, 34, 36, 38, 42, 44, 46 beyond the token sizes), four competing header scales (30, 28, 26, and a 24 override inside Home), and a floor of 11px micro text on faint gray that fails WCAG AA contrast. Figure 3.1 shows the histogram of explicit sizes against the token scale: the tallest bars sit off-scale.

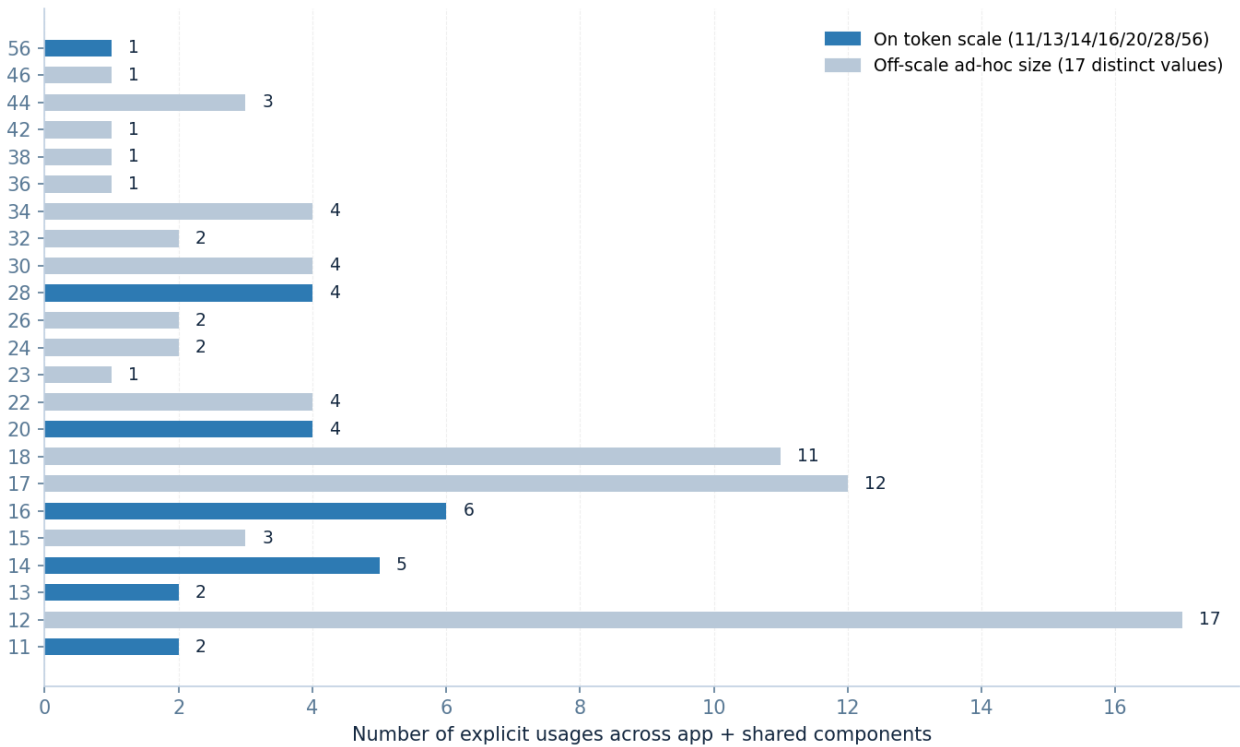


Figure 3.1 Explicit fontSize usages across app and shared components. Blue bars are token sizes; gray bars are ad-hoc values. 17 distinct off-scale sizes is the single largest source of visual drift.

Table 3.1 Proposed seven-step type scale (Chapter 4 uses it)

| Token     | Size / leading | Weight | Replaces (today)             |
|-----------|----------------|--------|------------------------------|
| display   | 56 / 60        | 800    | welcome 42-56, hero one-offs |
| title     | 28 / 32        | 700    | headers at 24, 26, 28, 30    |
| heading   | 20 / 26        | 600    | card titles at 18, 20, 22    |
| body      | 16 / 24        | 400    | 15, 16, 17, 18 body copy     |
| label     | 14 / 18        | 500    | 13px labels, button text     |
| caption   | 12.5 / 16      | 400    | 11px micro, 13px caption     |
| mono-data | 16 / 22 (tnum) | 600    | macro numbers in mixed sizes |

3.4 Icons: a differentiated asset, barely used

Nut AI ships its own stroke icon system: 53 named glyphs on a 24x24 grid in a 471-line Icon.tsx, rendered with react-native-svg at a 1.8 default stroke weight with a weight prop. This is rare and valuable; most apps ship whatever the icon library gives them, and none of them have a protein glyph or a thali bowl. But the set is under-deployed and undisciplined: only about 59 usages exist across the app, the shared Button, Field and Card primitives have no icon slot at all, several close affordances are raw text "x" characters while others use the close glyph, the FAB action grid reuses the identical search icon for two different actions, and the



training screens fall back to unicode text like plus signs and check marks inside button labels. Empty states and list rows are entirely text-only. Chapter 6 turns this asset into a system.

### 3.5 Components: five chip families and four Fields

The de-facto design system is Screen.tsx (216 lines exporting Screen, Label, Card, Button, Field, Row, useAction), plus Icon, Charts, DayTimeline, DayStatusControl, ConfidenceChip, CredentialForm and the onboarding kit. Around that core, screens have grown private copies. Figure 3.2 counts the duplication: five chip or pill implementations, four labeled-input Fields, four card containers, four header patterns, three progress rings, two primary button languages (a 48pt rectangular shared Button versus a 54 to 60pt pill hand-rolled in at least eight screens), two alert systems on web, and two copies of the ingredient-create form. Every duplication is a place where the next screen will drift one more pixel away from the last.

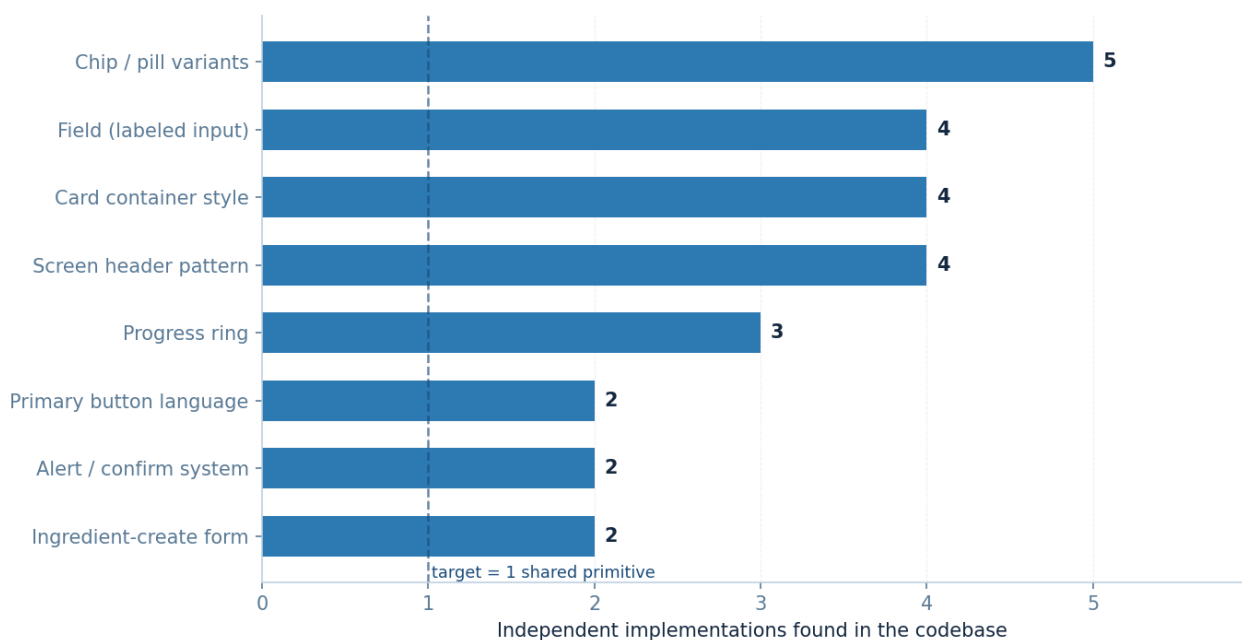


Figure 3.2 Component families with more than one independent implementation. The dashed line marks the target state: one shared primitive per family.

### 3.6 States, interaction and accessibility

Loading is handled by text placeholders ("Loading your day...") and `ActivityIndicator` in 11 files; there is not a single skeleton in the app. The assistant has no empty state at all, so first open shows a blank scroll area. Feedback is dominated by `Alert.alert`: 36 call sites across 12 files, plus a separate `window.confirm` path in `DayTimeline` that bypasses the styled web dialog shim, so two different confirm systems coexist on web. There are no toasts, no pull-to-refresh anywhere, and no bottom-sheet library: the FAB sheet is a fullscreen transparent modal with a fade, and result's Fix flow is a full-screen overlay view. Haptics exist only in onboarding (five selection ticks); logging a meal, completing a scan, or finishing a workout is silent. `react-native-reanimated 4.5.1` is installed and never imported, motion tokens are defined and unused, and `@expo/ui` is a dead dependency. Accessibility has a solid 216 `accessibilityLabel` and role usages to build on, but camera review pills run at 32pt height, the saved-foods "Log again" control is a `Text` with no `onPress` at

all, and the 11px faint text fails contrast on both themes.

|                              |                                |                                       |                           |
|------------------------------|--------------------------------|---------------------------------------|---------------------------|
| 36<br>Alert.alert call sites | 0<br>skeletons, toasts, sheets | 5 / 5<br>haptic sites, all onboarding | 2<br>dead UI dependencies |
|------------------------------|--------------------------------|---------------------------------------|---------------------------|

The verdict of this chapter is constructive: nothing here is architecturally wrong, and that is exactly why a design pass will compound. The token system exists, the icon set exists, the ally baseline exists, and the animation library is already paid for. The transformation is therefore mostly a campaign of consolidation, discipline and addition on foundations that are already poured.

## 4. Design Direction: Compose, Do Not Clone

The strongest products in this genre are compositions, not copies. The direction proposed here borrows deliberately: the navigation confidence of Spotify (five tabs, one thumb, no hamburger), the information density and calm chrome of Linear (small caps labels, tabular numbers, quiet borders), the spacing and large-title rhythm of Apple, the activity visualization warmth of Strava (rings, streaks, aggregate heatmaps), and a food-forward identity that is uniquely Nut AI: Indian-dish typography moments, macro colors that read like a thali palette, and the hand-drawn icon set no competitor has. The 2026 trend surveys converge on calm interfaces, purposeful motion instead of decorative animation, accessibility-first contrast, and AI-native transparency; this direction adopts all four as constraints rather than decorations.

### 4.1 Three theme directions

Direction A, Crystal Ink, is the recommendation: keep the ink neutral family and the existing macro colors, tighten and complete the semantic set, compute the dark palette for real, and add the missing elevation and state layers. It is the cheapest path because 80 percent of screens already consume the semantic theme; the work is filling gaps, not re-pointing screens. Direction B, Warm Bowl, moves neutrals toward warm paper tones with terracotta accents; it is distinctive and food-appropriate but would require touching every hardcoded ink reference and re-validating the charts. Direction C, Night Athlete, is a dark-first identity with a light secondary theme; it photographs beautifully for launch marketing but inverts the current entire light-first component set and doubles the contrast verification surface. All three are viable; A wins on risk-adjusted effort.

Direction A - Crystal Ink (recommended): evolve the existing ink neutrals, keep the macro identity colors

LIGHT



DARK



Figure 4.1 Direction A "Crystal Ink" in both modes: ink neutrals, semantic macro colors, and the existing affirm and safety pair. Values are the current tokens, completed rather than replaced.

Table 4.1 What each direction changes

| Direction                    | Neutrals                          | Accents                           | Cost                     | Risk                           | Verdict          |
|------------------------------|-----------------------------------|-----------------------------------|--------------------------|--------------------------------|------------------|
| A. Crystal Ink (recommended) | Keep ink ramp, fill semantic gaps | Keep macro trio, add state colors | Lowest                   | Low: known palette             | Ship this        |
| B. Warm Bowl                 | Warm paper grays                  | Terracotta + saffron              | High: all surfaces shift | Medium: chart + photo harmony  | Brand pivot only |
| C. Night Athlete             | Dark-first, light secondary       | Electric lime + blue              | Highest                  | High: every screen re-verified | Post-launch skin |

4.2 Typography decision

Two credible paths exist. Path one stays on the system font and enforces the seven-step scale from Table 3.1; it is a one-file change plus discipline, keeps the native feel, and adds zero kilobytes. Path two adopts Inter (or Geist) with tabular numerals enabled for all macro and weight numbers; it buys a noticeably more "designed" feel, perfect cross-platform metric consistency for the web build, and aligned digits in every ring, diary row and chart, at the cost of two font files and a font-loading step in expo-font. The recommendation is path two for one specific reason: this is a numbers app, and tabular numerals are the single highest-leverage typographic upgrade a data-dense product can make. Numbers stop jittering as values change, the scan result reads like an instrument, and the assistant's macro tables align. Inter's weights 400 to 900 also match the existing numeric-weight usage exactly, making the migration mechanical.

## 4.3 Spacing, radius, elevation and state layers

The 4pt spacing scale is kept as-is; violations are rare and the scale maps directly onto the shadcn-style spacing conventions discussed in Chapter 5. Radii gain one token: a 20pt "sheet" radius for bottom sheets, sitting between xl (24) and lg (16), matching modern sheet conventions. Elevation is the new layer: three shadow tokens (subtle for cards, medium for floating elements like the FAB and ActiveWorkout card, high for sheets and dialogs), plus a dark-mode strategy that substitutes border lightening for shadows since shadows are invisible on near-black backgrounds. Finally, a state layer completes the semantic set: pressed (6 percent ink overlay), disabled (38 percent opacity plus no shadow), selected (accent tint at 12 percent), and focus (2px accent ring for external keyboards on web). These four states, tokenized once, remove an entire class of inconsistency that shows up today as five different pressed behaviors.

### Design principle for every remaining chapter

One family, one primitive, one behavior. Any screen that needs a look the system does not offer must extend the system first and consume it second. This is the shadcn discipline adapted to Nut AI: own the code, compose from tokens, ship beautiful defaults.

## 5. The shadcn Way on Mobile

---

shadcn/ui describes itself precisely: not a component library, but how you build your component library. Its five published principles are Open Code (you own the component source and edit it directly), Composition (one predictable composable interface across all components), Distribution (a flat-file schema plus CLI to move components between projects), Beautiful Defaults (carefully chosen styles that look right out of the box), and AI-Ready (open code that models can read and extend). The docs count roughly 80 components, including the ones a nutrition app actually needs: Button, Card, Field, Item, Empty, Skeleton, Sheet, Drawer, Dialog, Toast, Progress, Tabs, Toggle Group, Chart, and a Message Scroller purpose-built for chat. The approach is proven at scale; the site carries the endorsements of OpenAI, Sonos and Adobe.

### 5.1 The React Native reality: react-native-reusables

shadcn/ui targets the web (React + Tailwind + Radix). The community port for this stack is react-native-reusables (RNR), which the repository bills as "bringing shadcn/ui to React Native": beautifully crafted components with Nativewind, open source under MIT, built on Expo, Radix web primitives and react-native-web so the same components run on native and web. The project is healthy: 8.7k stars, 45 contributors, 1,212 commits, rn-primitives at 1.4.0 as of March 2026, and fresh commits and documentation work in September 2026. Its existence proves the philosophy ports; it does not automatically make wholesale adoption the right move for an app that already ships.

### 5.2 Two adoption strategies, one decision

Strategy one is full adoption: add Nativewind, pull RNR primitives, and rewrite screens against Tailwind-class components. This buys the exact shadcn look, the fastest future component flow, and one mental model across web and native. It also means re-touching all 52 screens, replacing the existing token

context with Tailwind CSS variables, adding a build step to the pipeline, and re-testing the 44-screen surface on both platforms. Strategy two is the pattern port: keep the StyleSheet token engine, but adopt the shadcn component inventory, API shapes and defaults as the specification for a rebuilt Nut AI primitive layer, and pull in battle-tested libraries only where the ecosystem is strictly better (a bottom sheet, a toast queue, skeleton placeholders). Strategy two is recommended. It captures the design payoff, respects the existing test suite, avoids a Nativewind learning curve mid-product, and can ship wave by wave because it never forks the rendering model.

**Table 5.1 shadcn inventory mapped to Nut AI equivalents**

| shadcn component      | Nut AI today                                | Action                                      |
|-----------------------|---------------------------------------------|---------------------------------------------|
| Button (+ icon slot)  | Screen.Button, text-only; 2nd pill language | Rebuild one Button, 48/56pt, icon slot      |
| Card / Item           | 4 card dialects                             | One Card + Item row primitive               |
| Field (label + input) | 4 Field copies                              | One Field, error + hint slots               |
| Sheet / Drawer        | None (fullscreen modal FAB)                 | Bottom-sheet lib + shared Sheet             |
| Toast / Sonner        | 36 Alert.alert sites                        | Toast queue; alerts only for destructive    |
| Skeleton              | None (spinners + text)                      | Skeleton primitive + shimmer                |
| Empty                 | Mostly missing or text-only                 | Empty primitive: icon + title + action      |
| Dialog / Alert Dialog | alert-web shim + window.confirm             | One Dialog; kill window.confirm path        |
| Tabs                  | Custom pill tabBar + 5 chip sets            | Tabs pattern reused for chips               |
| Toggle Group          | Segmented in onboarding only                | Generalize to app-wide control              |
| Progress              | 3 ring implementations                      | One ProgressRing (stroke + track)           |
| Tooltip / Info        | Explainer paragraphs inline                 | Info icon + popover on web                  |
| Message Scroller      | assistant.tsx custom scroll (no autoscroll) | Adopt pattern: autoscroll + anchor          |
| Chart                 | Charts.tsx Line/Bar                         | Keep; add donut + area styles               |
| Badge / Chip          | 5 chip implementations                      | One Badge/Chip with variants                |
| Spinner               | ActivityIndicator x11                       | Replaced by skeletons; Spinner for >10s ops |

## 5.3 Migration order

The migration rides the wave plan in Chapter 13. Wave one lays the token foundation (the shadcn "theming" step), since every primitive consumes tokens. Wave two rebuilds the primitive layer in place: Button, Card, Field, Badge, Skeleton, Empty, Sheet, Toast, Dialog, each replacing its scattered copies screen by screen, with the duplication census in Chapter 12 as the worklist. Wave three re-skins the six flows on the new primitives. The acceptance test for "shadcn way adopted" is simple to state: no screen may define its own

control; every visual decision resolves to a token; and deleting any single primitive file breaks the build, because everything real uses it.

#### **AI-ready is not a slogan here**

Because Nut AI is built AI-first (scan, assistant, correction flows), the open-code principle pays twice: components whose code is small, token-driven and composable are exactly the surface an AI coding agent can extend safely. A one-primitive-per-family app is the cheapest app to iterate on with AI.

## **6. Iconography System**

---

Icons are the fastest credibility signal an interface sends, and the cheapest to get wrong. Nut AI already owns the hard part: 53 original stroke glyphs drawn on a 24x24 grid, including domain metaphors (protein, carbs, fat, fiber, barcode, nutrition label, receipt, dumbbell, muscle, lotus, bowl) that no generic library carries. The audit found the set under-used (59 usages), inconsistent at the edges (text "x" closes, unicode check marks in buttons, a duplicated search icon in the FAB grid), and invisible in the places that need it most: the shared Button, Field and Card primitives expose no icon slot, and every training-screen empty state is bare text.

### **6.1 Rules borrowed from Apple**

The Apple HIG icon guidance translates directly into five rules for Nut AI. One concept per icon: a glyph must communicate a single idea instantly, with familiar metaphors, which the custom set mostly achieves and the rarely used ones (scaleBalance, dot1/3/6) should be re-audited against. Consistency of size, detail and stroke: every glyph must share one grid and one stroke family, and the stroke must visually match the weight of adjacent text, heavier next to bold labels, lighter next to captions. Optical centering: asymmetric glyphs get pixel nudges, not geometric centering, so a download or flame glyph does not look off-axis. Inclusive imagery: human figures stay gender-neutral (the existing male, female and nonbinary trio is correct). Accessibility: every glyph carries a label when it acts alone, and icons ship as vectors that scale cleanly to the three semantic sizes.

### **6.2 Keep custom, align to Lucide, fill gaps**

Three sourcing options were evaluated. Option one, full library adoption: lucide-react-native (the shadcn default, a Feather fork with several times the coverage, MIT, tree-shakeable) would replace the custom set; rejected because it throws away the domain glyphs that differentiate the brand. Option two, Phosphor (six weights, the widest style range) as an addition; rejected because two stroke philosophies in one app is exactly the inconsistency this report is eliminating. Option three, recommended: keep the custom set as the system of record, align its grid and stroke conventions to Lucide (24px grid, 2px base stroke, rounded caps) so any Lucide glyph can be imported as a gap filler without visual clash, and extend Icon.tsx with a lucide passthrough that maps missing metaphors (share, crown for premium, sparkle for AI, flame-variant for streaks) into the same Icon API. The icon component gains three semantic sizes (24 default, 20 dense, 16 inline), a mandatory label when rendered without adjacent text, and color resolved exclusively from the theme.

**Table 6.1 Core action-to-icon mapping (excerpt of 20)**

| Action / surface        | Icon                      | Source      | Notes                      |
|-------------------------|---------------------------|-------------|----------------------------|
| Tab: Home               | home                      | custom      | active: filled accent tint |
| Tab: Food               | bowl                      | custom      | signature glyph            |
| Tab: Train              | dumbbell                  | custom      | unchanged                  |
| Tab: Progress           | chart                     | custom      | align stroke to 2px        |
| Tab: Profile            | person                    | custom      |                            |
| Scan food (FAB)         | scan                      | custom      | hero action, 26pt          |
| Log exercise            | run                       | custom      | replaces text-only tile    |
| Saved foods             | bookmark                  | custom      | replaces duplicate search  |
| Recipes                 | book-open                 | lucide fill | gap fill                   |
| Food database           | search                    | custom      | single use now             |
| AI assistant            | sparkles                  | lucide fill | AI affordance              |
| Log a meal (row action) | plus-circle               | custom      | 44pt target                |
| Favorite meal           | heart                     | custom      | fills on active            |
| Save as usual meal      | bowl-plus                 | custom new  | merges 3 save variants     |
| Edit grams / quantity   | pencil                    | custom      |                            |
| Delete item             | trash                     | custom      | confirm via dialog         |
| Undo / redo             | uturn-back /<br>uturn-fwd | lucide fill | HIG standard action        |
| Show thinking (AI)      | chevron-down              | custom      | rotates when open          |
| Close sheet / modal     | close                     | custom      | replaces all text x        |
| Streak / consistency    | flame                     | custom      | paired with count          |

## 6.3 Deployment targets

The discipline lands in four places. First, primitives: Button, Field, Badge and Empty grow icon slots so shared UI can carry glyphs, which alone multiplies icon coverage without touching screens. Second, the FAB grid: six distinct glyphs replace the duplicated pair, converting the tile sheet into a scannable menu. Third, the training screens: every unicode label (check marks, plus signs, move-up text) becomes a glyph plus label pair, and row actions move into a context menu with icons. Fourth, empty and error states across all 52 screens get the Empty primitive with an icon, which is where the set stops being decoration and starts being navigation. Success metric: icon-annotated actions rise from roughly 59 usages to above 250, and zero

user-facing text glyph substitutes remain.

## 7. Navigation and Information Architecture

Nut AI’s information architecture is fundamentally right: five tabs, scan as the hero action, journal screens one level deep. The transformation therefore polishes rather than restructures. The tab bar keeps its five destinations but upgrades its craft: 12pt minimum labels (fixing the 11px AA failure), active state rendered as an accent-tinted pill with a 120ms morph instead of a flat background swap, and haptic selection on switch. The FAB, currently food-tab only, becomes the global create action on all five tabs, opening a true bottom sheet (not the fullscreen modal tile grid) with six grouped actions, each with its own glyph per Table 6.1, an escape hatch by tap-outside and swipe-down, and the scan action visually promoted as the sheet’s primary row.

### 7.1 Fixing the Home / Food duplication

The deepest IA question is the Home and Food tab overlap: DayTimeline mounts in both, the food tab re-lists quick actions that mirror the timeline’s row actions, and users meet the same component at two very different densities. The proposal sharpens each tab’s single job. Home becomes the read surface: hero ring, macro summary, adaptive target card, and the day timeline in its full form. Food becomes the write surface: a large "Log food" entry, the shortcut strip (Recent, Frequent, Favorites, Usual, Saved) expanded into one-tap logging cards with thumbnails and gram presets, saved meals, recipes, and the Indian dish library; the second timeline copy is removed entirely. This kills the duplicated component, halves the food tab’s render cost, and gives each tab a reason to exist.

Table 7.1 Navigation decisions at a glance

| Surface        | Today                                   | Proposed                                                    |
|----------------|-----------------------------------------|-------------------------------------------------------------|
| Tab bar        | Custom pill bar, 11px labels            | Same structure; 12pt labels, accent morph, selection haptic |
| FAB            | Food tab only; fullscreen modal grid    | All tabs; bottom sheet, scan promoted, distinct glyphs      |
| Home tab       | Ring + timeline + adaptive card         | Read-only day surface; timeline stays                       |
| Food tab       | Quick actions + chips + second timeline | Write surface: log entry, shortcuts, saved, recipes         |
| Headers        | 4 hand-rolled variants (30/28/26/24)    | One Screen header, iOS large-title collapse                 |
| Assistant      | Card push (reads as modal)              | True modal presentation, matching its role                  |
| Deep links     | nutai:// declared, no handler           | Router-level handler: widget + notification taps            |
| Active workout | Floating card, 1s poll, bottom 92       | Progress banner with set count, reanimated slide-in         |
| Onboarding     | 22 screens, slide push                  | 12 screens, single progress rail (Ch. 8)                    |



## 7.2 Header unification and modal hygiene

Every screen currently hand-rolls a header row, producing four dialects of the same object: the Screen.tsx 30pt back-button header, token 28pt titles, the assistant's 26pt custom header with close button, and Home's 24pt override. One Screen header is specified instead: an iOS-style large title (28pt bold) that collapses to a 17pt inline title on scroll with a hairline border, a left slot for back or close, and a right slot for one or two context actions as icons. Modal hygiene follows the same logic: the 20 registered modals are audited so that focus experiences (assistant, checkin, camera, result) present as full-screen modals with close glyphs, while utility screens keep card pushes with back chevrons, and the deep-link handler registered at the router root finally delivers the widget and notification taps the scheme promises. The ActiveWorkout floating card is rebuilt as a slim progress banner that slides in with reanimated, shows completed-set count instead of polling blindly every second, and deep-links into the live workout.

## 8. Screen-by-Screen Transformation Blueprints

This chapter is the core of the report: per flow, the problem today, the redesign, and the pattern it draws from. Each blueprint assumes the system chapters already landed (tokens, primitives, icons, navigation), so every screen change is expressed as composition, not new art. Figure 8.1 summarizes the consolidation effect across flows.

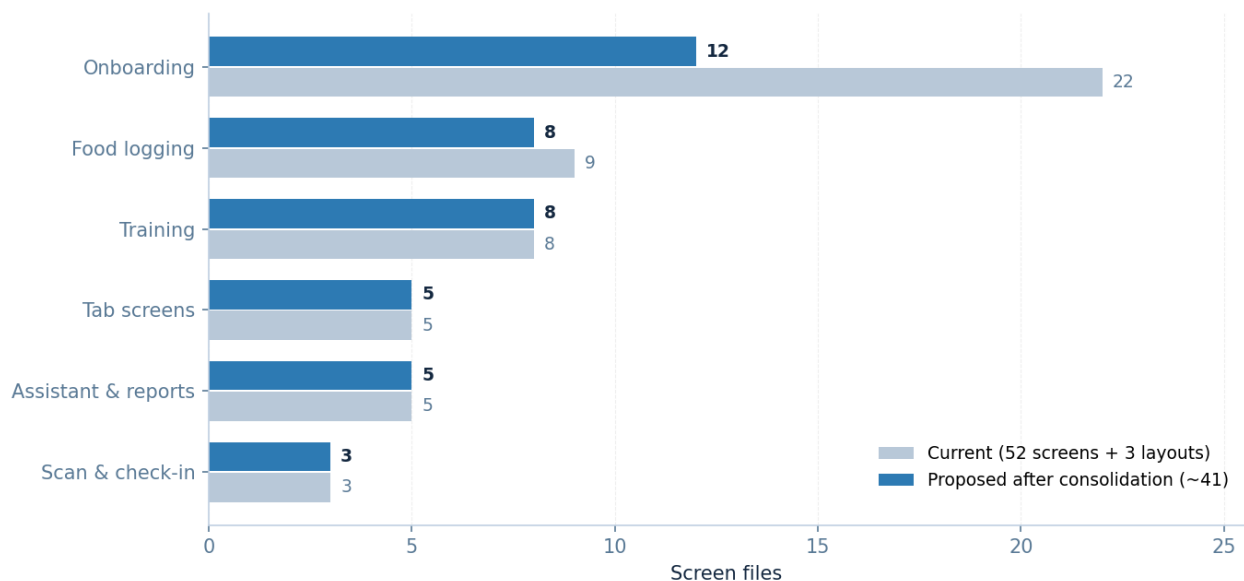


Figure 8.1 Screen inventory today versus after onboarding consolidation and Food-tab deduplication. Training, tabs and scan keep their screen counts but change internally.

### 8.1 Onboarding: 22 screens to 12

Today's flow is 20 defined steps plus plan and restore, and five of them are pure motivational interstitials (thanks, generate, notifications, potential, trend) that add taps, not information. The redesign keeps the celebrated foundations, the shared Chrome, the ruler and wheel pickers with their selection haptics are genuinely excellent, and cuts to twelve screens: welcome (with a live demo of the scan instead of a tilted

phone mock), sex and workouts merged into one activity card group, diet and blocker merged, accomplish, body metrics on one combined picker screen, desired weight, provider and key on one screen, health permissions, one consolidated projection screen (trend and potential fused into a single chart moment), rollover, plan reveal, and notifications. The single progress rail from Chrome persists across all steps so users always know the distance left. The plan reveal, already the heaviest custom screen at 353 lines, becomes the hero moment: numbers count up, the breakdown expands with a spring, and a success haptic marks the finish. Pattern references: goal-selection flows in Mobbin-class onboarding sets and the Cal AI pattern of showing product value before asking for setup.

## 8.2 Home: an instrument, not a dashboard

Home already has the right skeleton (streak pill, 56-day strip, calorie ring with overflow arc, three macro cards, adaptive-target card, timeline). The craft pass makes it read like an instrument. The hero ring gains a count-up on focus and a press-for-detail expansion showing remaining calories by meal slot; numbers set in tabular figures stop jittering as the day updates. Macro cards become compact stat rows with per-macro rings inline, freeing vertical space for the timeline. The adaptive-target card gets an explicit state machine (stable, adjusting, locked after check-in) so its copy stops reading like a warning. The 56-day strip keeps its 44pt rows and gains today-pulse and future-dimmed states. Skeletons replace the "Loading your day" text, and pull-to-refresh syncs health data with a satisfying snap. Reference patterns: Strava's aggregate visualization, MacroFactor's calm target cards, Apple Fitness ring language.

## 8.3 Food logging: from forms to flow

Logging is the most performed action, so it gets the most redesign. The Food tab's quick actions become an icon grid (Chapter 7), and the shortcut strip becomes one-tap log cards: each shows the meal, its last gram preset, and a plus, so a repeat log is a single tap with a success haptic, undo toast anchored. The 1,130-line food-search screen is decomposed into a search-first surface (query, FTS results, recent chips) with the unknown-dish decomposer and the on-the-spot ingredient creator moved behind clear entry points instead of stacked inline sections; both share the one rebuilt Field and ingredient form (killing the dish-composer duplicate). Food-review becomes a stepper: quantity and unit confirmed in one card with large touch ladders, slot picker as a segmented row, and a sticky summary footer with the log button. Saved-foods gets the Empty primitive and its dead "Log again" text replaced by a working primary button. Meal-detail adopts the row-action pattern (swipe or long-press for edit, duplicate, remove) instead of per-row text buttons. References: Hevy's fast repeat logging, Cal AI's photo-first flow, Mobbin food-review steppers.

## 8.4 Scan and result: confidence you can see

The camera screen's four mode pills and review toggle are kept, rebuilt on the Badge and Segmented primitives with proper 44pt targets (the review pills run 32pt today), and the viewfinder gains a subtle capture-guide arc that doubles as the shutter animation. The analyzing state is where this flow wins trust: a skeleton ingredient list builds while the model works, replacing the model caption ("Scanning with ...") which moves to settings diagnostics where it belongs; users need confidence, not vendor names, mid-scan. The result screen (1,091 lines) keeps quick and advanced modes; confidence chips get a legend on first appearance, uncertain rows show the reason inline instead of behind a tap, and the Fix flow collapses from a

three-stage overlay into one bottom sheet with a note field and before-and-after rows. Add-ingredient becomes a bottom sheet with recents. The failed state gets a real Empty illustration, the specific reason, and two actions: retake or log manually. References: Cal AI's instant-result theater done honestly, Cal AI and Hevy reviews consistently praise fast, legible results.

## 8.5 Training: the Hevy pattern

Training is where the app's UX debt is most concentrated: workout.tsx packs roughly twelve equal-weight text buttons per exercise card (select for circuit, notes, rest edits, skip rest, move up, replace, remove, ungroup, save as routine, plate helper, duplicate set, delete set), the tab file itself is minified one-line code, and set logging is a form rather than a table. The redesign adopts the pattern that made Hevy the best-reviewed tracker in its class: a set table where each row is number, previous, weight, reps, and a check that completes the set with a satisfying haptic and auto-advance; exercise rows collapse to name plus set count with a context menu (long-press or chevron) holding the twelve secondary actions in icon-labeled groups; the rest timer lives as a persistent chip that counts down and pulses at zero; supersets show as linked color-coded groups using one tinted token, not four ad-hoc alphas. The equipment plate calculator keeps its function, restyled as a bottom sheet. log-exercise's internal four-screen wizard compresses into two: pick (search, filters, recents) then configure (sets, reps, weight in one card). References: Hevy set-table logging, Strong's rest chip, Mobbin workout-history sets.

## 8.6 Progress and reports

Progress is the analytics tab and already leans on local Card and Chip primitives that the system pass will absorb. The upgrade is hierarchy: one hero metric per window (weight or intake, user-selected), rendered large with tabular numerals; secondary charts in a consistent card rhythm with the shared Line and Bar charts; period chips as one Toggle Group; and the weekly and monthly report cards visually unified with the assistant's card language so reports feel like a continuous narrative. Empty windows get the Empty primitive with a straight explanation of what fills them. Reference patterns: MacroFactor's expenditure charts and Linear's calm data density.

## 8.7 Assistant: a conversation that feels engineered

The assistant already has streaming, reasoning disclosure and tool cards; the pass makes it feel intentional. Message list adopts the Message Scroller pattern: automatic scroll-to-new-reply (fixing the audit's no-autoscroll finding), keyboard-safe insets, and an anchored scroll-to-bottom pill when the user scrolls up mid-stream. Bubbles get the system's radius and tone: user right in accent tint at 12 percent, assistant left on card background, with the model name appearing once at conversation start, never per message. The reasoning disclosure becomes a clean collapsible with the chevron glyph, tool cards unify into the Item primitive with status badges (proposed, applied, failed), and the first-open experience gets an Empty state with three starter prompts. Word-reveal streaming stays, but raw JSON guards become silent states rather than visible ones. References: shadcn Message Scroller, modern AI chat patterns from the 2026 surveys (AI-native transparency).

## 8.8 Profile and settings: progressive disclosure

Profile is 395 lines and eight sections, including three multi-sentence caption paragraphs, license attributions and the medical disclaimer, all at one scroll level. The redesign uses iOS grouped-list architecture with sub-pages: Goals, AI Provider (with the reseller and model fields behind an "Advanced" disclosure), Units and Health, Data (export, import, erase with destructive confirm dialogs), About (folding licenses and the disclaimer), and a diagnostics page that becomes the new home for the debug-class information this report removes from user surfaces: active scan and chat model identifiers, base URL, and webhook status. Each settings row uses the Item primitive: icon, label, current value in muted text, chevron. The result scans in half a screen and every power option remains two taps away. Reference patterns: SaaSFrame settings structures, iOS grouped lists.

## 9. Interaction, Motion and Haptics System

Nut AI defines five motion durations in tokens.ts (90, 160, 240, 380, 520 milliseconds, scaled by a reduce-motion factor) and ships zero custom animation: react-native-reanimated 4.5.1 sits unused, gesture-handler wraps the app but powers nothing, and the only choreography is native stack transitions. The 2026 trend consensus is explicit that motion must be purposeful rather than decorative; this chapter gives every animation a job. The library decision is already made by the package.json: enable reanimated, keep gesture-handler for sheets and swipes, and standardize on springs for anything position-based and durations from the token set for anything opacity-based. Every behavior below respects the existing motionScale reduce-motion hook, collapsing to instant states when the user opts out.

Table 9.1 Motion specification

| Behavior               | Spec                                       | Job it does                          |
|------------------------|--------------------------------------------|--------------------------------------|
| Press feedback         | scale 0.97 + dim 6%, 120ms out             | Confirms touch on every Pressable    |
| Tab switch             | accent pill morph 240ms + selection haptic | Orients within the tab set           |
| FAB sheet              | spring, translateY, sheet radius 20        | Spatial origin of create actions     |
| Ring / count-up        | 600ms ease-out on focus                    | Draws eye to the day's hero number   |
| Skeleton shimmer       | 1.2s loop, 20% sweep                       | Communicates structured loading      |
| Log success            | check morph 160ms + success haptic         | Rewards the core action              |
| Stream reveal          | word fade-in 90ms cadence                  | Signals live AI work                 |
| Banner slide (workout) | spring from bottom, drag to dismiss        | Non-blocking resume affordance       |
| Pull-to-refresh        | native RefreshControl                      | Expected sync gesture                |
| Modal transitions      | native sheets/slides only                  | Never invent custom page transitions |

### 9.1 Haptics map

The Apple HIG taxonomy gives a complete vocabulary: notification patterns (success, warning, error) for outcomes, impact patterns (light, medium, heavy, rigid, soft) for physical metaphors, and selection for value changes. The app already uses selection five times, all inside onboarding controls; the map extends it app-wide with one rule from the guidelines: a haptic must reinforce a cause-and-effect relationship, and never the same pattern for opposite meanings. Logging a meal lands success; completing a set lands a light impact with success at workout finish; picking a slot or a chip lands selection; destructive confirmation lands warning; failures land error. Haptics complement visual feedback, never replace it, stay short, and remain optional (a settings toggle, default on, following the make-it-optional rule). expo-haptics is already a dependency; this is a wiring task measured in hours.

**Table 9.2 Haptic events app-wide**

| Event                                | Pattern                | Source of truth                       |
|--------------------------------------|------------------------|---------------------------------------|
| Picker tick, chip toggle, tab change | Selection              | Existing onboarding pattern, extended |
| Log meal / log weight                | Success (notification) | Core reward moment                    |
| Complete a set                       | Light impact           | Fast, physical, repeatable            |
| Workout finished                     | Success                | Milestone                             |
| Scan captured                        | Light impact           | Shutter metaphor                      |
| Delete / erase confirm               | Warning                | Destructive intent                    |
| Action failed                        | Error                  | Negative outcome                      |
| Sheet open / close                   | None                   | Motion carries it; avoid noise        |

## 9.2 Skeletons: the NN/g decision rules

Nielsen Norman Group's research gives precise placement rules this report adopts wholesale. Under one second: show nothing, indicators flash and annoy. One to ten seconds: full-page loads use skeleton screens that mimic the coming layout (they reduce perceived wait and build a mental model); individual modules use spinners. Over ten seconds: a progress bar with an explicit estimate. Frame-only skeletons (chrome with no content placeholders) are explicitly not recommended and are not used here. Animated shimmer is applied gently (one slow sweep, paused under reduce-motion) because NN/g flags aggressive animation as an accessibility risk. Per screen: Home and Food get timeline skeletons; search gets result-row skeletons; scan-analyzing gets the ingredient skeleton list; assistant gets a three-line reply skeleton when the first byte is slow; progress and reports get chart-shaped skeletons. The 45-second scan and streaming paths keep spinners plus textual progress because they exceed the skeleton window.

## 10. States and Feedback Overhaul

A screen is only as good as its worst state. The audit found strong coverage of happy paths, text-placeholder loading, one missing empty state (the assistant's first open), two competing confirm systems, and thirty-six alert call sites doing the work of a notification system. The overhaul defines four states for every screen, then rebuilds the feedback layer once so all screens inherit it.

**Table 10.1 State matrix per flow**

| Flow               | Loading                          | Empty                                  | Error                               |
|--------------------|----------------------------------|----------------------------------------|-------------------------------------|
| Home               | Timeline skeleton                | Empty-day card with log actions        | Inline retry banner                 |
| Food / search      | Result-row skeletons             | No-results Empty with query help       | Retry + offline note                |
| Scan result        | Ingredient skeleton list         | Failed Empty: reason + retake / manual | Classified reason, not blanket text |
| Training           | Workout skeleton                 | Create-first Empty                     | Per-action inline errors            |
| Progress           | Chart-shaped skeletons           | Window Empty with explanation          | Retry card                          |
| Assistant          | Reply skeleton (slow first byte) | Welcome Empty + starter prompts        | Inline error bubble + retry         |
| Settings / profile | Instant (local)                  | Not applicable                         | Inline field errors                 |

### 10.1 One toast system, two rules

The toast system (sonner-native pattern: queued, top-anchored, swipe-dismissable, one visible at a time) becomes the default feedback for reversible outcomes: meal logged with an Undo action, recipe saved, goal updated, export prepared. Two rules govern it. Rule one: toasts confirm success and offer the next action; they never ask questions. Rule two: alerts are reserved for destructive or irreversible confirmations (erase everything, delete meal, reset onboarding), which collapses 36 Alert.alert sites to roughly eight justified ones, removes the web shim's reason to exist in most flows, and retires the DayTimeline window.confirm bypass by replacing it with the shared Dialog. Inline validation stays inline (Field error slots), and every useAction failure surfaces as inline error plus retry, never as a silent swallow like today's Log-it path.

### 10.2 Optimistic logging and honest uncertainty

Two trust upgrades complete the layer. Optimistic logging: because the repo layer already supports undo operations, a log can apply instantly with an undo toast rather than waiting on any round trip, matching the fast-repeat pattern users praise in Hevy. Honest uncertainty: the existing ConfidenceChip system extends so every AI-supplied number carries its uncertainty visually (chip tone, reason line, and a tap through to the why), and error classifications stop collapsing distinct failures into one message; the scan chain's known catch-all mislabels are fixed at the source so the UI can narrate truthfully. Feedback that respects the user's intelligence is a feature; this layer is where the app's offline-first honesty becomes visible.

# 11. Accessibility and Visual Trust

Accessibility is already half-built: 216 accessibilityLabel and role usages, a lint-enforced 44pt tap-target constant consumed in 15 files, chart roles with summaries, and freezeOnBlur keeping hidden tabs out of the tree. The failures are specific and fixable. The 11px micro style on textFaint (#8A8A99 on white, roughly 3.5:1) fails AA wherever it appears (inactive tab labels, timestamps, model captions) and is retired by the new type scale (12.5px floor, darker muted token). The dark palette’s own comment admits it was never computed; Chapter 4’s direction A includes computing every pair and regenerating the ramp. The #ccc borders in assistant cards vanish in dark mode, and the Tailwind alpha tints in the training screens (#10b98120 family) carry almost no contrast on near-black; both move to token tints. Camera review pills at 32pt and progress chips at 40pt join the saved-foods dead control on the violations list, all repaired by primitives with targets baked in.

Table 11.1 Contrast and target fixes

| Issue                   | Where                            | Fix                                          |
|-------------------------|----------------------------------|----------------------------------------------|
| 11px textFaint 3.5:1    | Tab labels, timestamps, captions | 12.5px floor + darker muted; AA computed     |
| Dark palette unverified | tokens.ts dark block             | Compute all pairs; regenerate ramp; CI check |
| #ccc borders in dark    | AssistantCards                   | Border token, theme-resolved                 |
| Alpha Tailwind tints    | programs, equipment, routines    | Selected-state tint tokens                   |
| 32-40pt targets         | camera pills, progress chips     | 44pt in primitives; hitSlop only for icons   |
| Dead Log again control  | saved-foods.tsx:69               | Working primary button                       |

## 11.1 Dynamic type and VoiceOver plan

Two structural investments finish the job. Dynamic type: allowFontScaling is enabled with the new type scale mapped to accessibility tiers, and layouts audited for the 130 percent case (the ring cards and set table get explicit scale-down rules for numerals while labels grow). VoiceOver: existing labels are kept and completed on the new primitives (every icon-only control labeled, every sheet announced, live regions for streaming replies and log confirmations), and the custom charts’ image roles gain value summaries. Trust closes the chapter: uncertainty stays visible (Chapter 10), the medical disclaimer remains reachable from About, and contrast verification becomes a CI gate so the [VERIFY] comment never has to exist again. Inclusive design here is not a compliance exercise; it is the same clarity that benefits every user in sunlight, one-handed, at a glance.

## 12. Bloat Removal Census

Design systems improve by subtraction first. This census catalogs every element the audit flagged as dead, duplicated, debug-class or mergeable, organized as kill, merge and hide lists so each item has a disposition. Figure 12.1 shows the distribution; the tables name every item. Nothing here removes user capability: every deletion either removes something that never worked, consolidates duplicates of the same function, or moves developer information to where developers look for it.

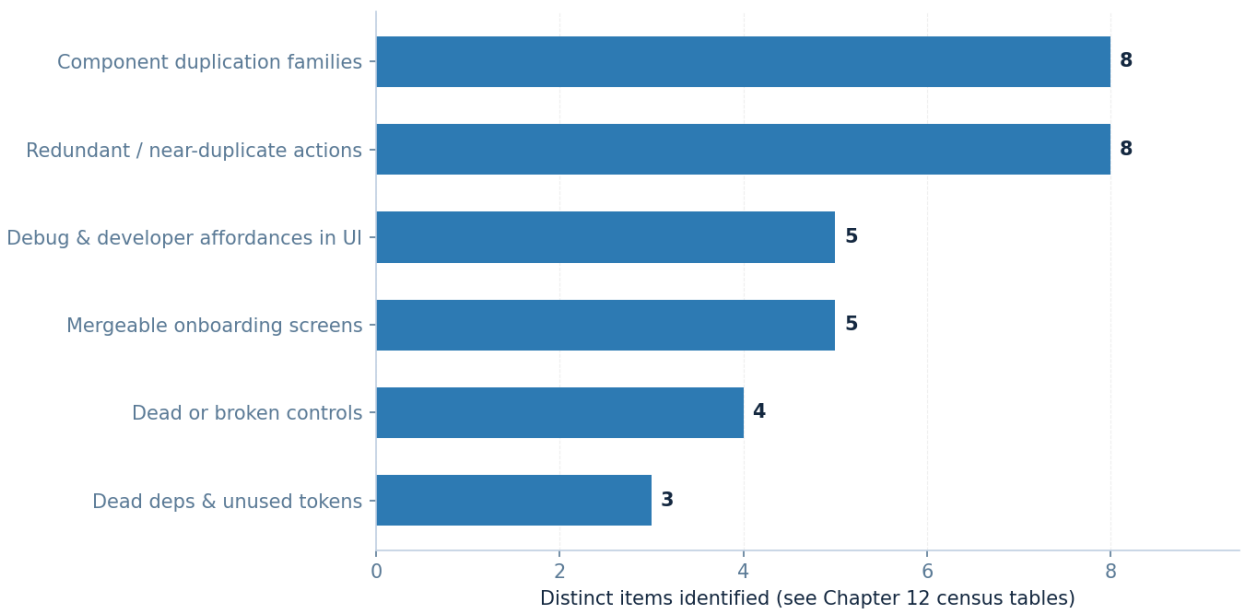


Figure 12.1 Bloat census by category. Component duplication and redundant actions dominate; both collapse into the Chapter 5 primitives.

Table 12.1 KILL list (remove outright)

| Item                            | Location                                            | Why                                                   |
|---------------------------------|-----------------------------------------------------|-------------------------------------------------------|
| Dead "Log again" text           | saved-foods.tsx:69                                  | Not pressable; the screen's only action is decorative |
| window.confirm path             | DayTimeline.tsx                                     | Bypasses the styled web dialog; second confirm system |
| Text "x" closes, unicode glyphs | camera, assistant, training screens                 | Inconsistent affordances; replaced by icons           |
| "Active scan model" caption     | provider-settings.tsx                               | Debug info in user UI; moves to diagnostics           |
| "Scanning with" model caption   | result.tsx                                          | Vendor names mid-scan; moves to diagnostics           |
| Duplicate FAB search tile       | tabs _layout action grid                            | Two actions, one icon; distinct glyphs                |
| 1s polling in ActiveWorkout     | ActiveWorkout.tsx                                   | Battery cost for a static pill; event-driven          |
| Dead deps and tokens            | reanimated (until enabled), @expo/ui, motion tokens | Enable reanimated; delete @expo/ui                    |



**Table 12.2 MERGE list (one primitive each)**

| Family                    | Copies today                                 | Merged to                                          |
|---------------------------|----------------------------------------------|----------------------------------------------------|
| Save actions per meal row | 5 (favorite, usual, save meal, repeat, copy) | 2: Favorite + Save as usual                        |
| Primary button languages  | 2 (48pt Button vs 54-60pt pill)              | One Button, two sizes                              |
| Chip / pill sets          | 5 implementations                            | One Badge with variants                            |
| Field (labeled input)     | 4 copies                                     | One Field with error + hint                        |
| Card containers           | 4 styles                                     | One Card + Item row                                |
| Progress rings            | 3 (home, rollover, welcome)                  | One ProgressRing                                   |
| Header patterns           | 4 variants                                   | One Screen header, large title                     |
| Alert / confirm systems   | 2 on web                                     | Toast + Dialog, platform-native alerts only native |
| Ingredient-create forms   | 2 (food-search, dish-composer)               | One shared form component                          |
| DayTimeline mounts        | 2 with divergent flags                       | 1 on Home; Food tab drops it                       |

**Table 12.3 HIDE or FOLD list (progressive disclosure)**

| Item                             | Today                              | Disposition                               |
|----------------------------------|------------------------------------|-------------------------------------------|
| Reseller / custom model fields   | Always visible, 4 free-text fields | Behind Advanced disclosure in AI Provider |
| Profile 8 sections               | One long scroll                    | Grouped list with sub-pages (Ch. 8.8)     |
| Onboarding interstitials         | 5 low-information screens          | Fused into 2 moments (Ch. 8.1)            |
| Workout 12+ row actions          | Equal-weight text buttons          | Context menu groups (Ch. 8.5)             |
| Fix-result three-stage flow      | Full-screen overlay chain          | One bottom sheet                          |
| License attributions, disclaimer | Inline in Profile                  | About sub-page                            |

The census is deliberately conservative: each row cites its audit evidence, and the merge rows depend on the Chapter 5 primitives landing first so replacements exist before deletions. Executed in order, the kill list alone removes visible debt within a day, and the merge list converts the duplication chart of Figure 3.2 into a wall of ones.

## 13. Implementation Roadmap

The transformation ships in four waves, ordered so trust is won early: every wave leaves the app fully releasable, and no wave depends on a rewrite. Effort estimates assume the existing one-engineer-plus-AI workflow with the 733-test gate held green throughout. Each wave has a definition of done expressed as checks, not vibes, so progress is auditable the same way the QA report was.

**Table 13.1 Four waves**

| Wave                | Scope                                                                                                                                       | Effort    | Done when                                                                                          |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------------|
| 1. Foundation       | Type scale + contrast fixes; kill list; toast + dialog; skeleton + empty primitives; haptics map; icon consistency pass; header unification | 2-3 days  | Zero off-scale sizes; zero text-glyph buttons; 36 alerts collapsed; skeletons on 6 core screens    |
| 2. Component layer  | Button, Card, Field, Badge, Item, ProgressRing, Sheet; FAB sheet; tab bar craft; motion enable (reanimated); states matrix                  | 4-6 days  | Figure 3.2 families at one implementation each; bottom sheets live; press + haptic on all controls |
| 3. Screen rebuilds  | Onboarding to 12; scan/result; training Hevy pattern; assistant polish; Food tab write surface; settings sub-pages; diagnostics page        | 8-12 days | Ch. 8 blueprints shipped; screen count 52 to ~41; model captions only in diagnostics               |
| 4. System hardening | Dark palette computed + CI contrast gate; dynamic type pass; deep-link handler; icon set extension; primitive docs; dead dep removal        | ongoing   | CI all y gate green; nutai:// routes; component library documented                                 |

Sequencing notes. Wave one is pure subtraction and small additions; it needs no primitive dependencies and should land as one reviewable series. Wave two is the only wave with architectural surface (the primitive rebuild), and it rides the duplication census as its worklist, one family per commit. Wave three is parallelizable across flows once primitives exist; onboarding and training are independent. Wave four has no fixed end: the contrast CI gate and the component docs are what keep the system from drifting again, which is the failure mode that produced the 17-size type scale in the first place. The metric that matters at the end is the same one this report began with: the distance between what the app knows and what the app feels like, which this plan closes to zero.

### Definition of done for the whole program

A new screen can be built entirely from primitives and tokens with zero local styling; every interactive element meets 44pt with a press state and a haptic; every list has loading, empty and error states; every color pair passes AA in both themes as verified by CI; and the icon audit finds no text glyph standing in for a symbol anywhere.

# Appendix

## A Source bibliography

shadcn/ui Documentation: Introduction, Components, Theming. [ui.shadcn.com](https://ui.shadcn.com) (accessed September 2026).

React Native Reusables: Bringing shadcn/ui to React Native. [founded-labs/react-native-reusables](https://github.com/founded-labs/react-native-reusables), GitHub, MIT, rn-primitives 1.4.0 (March 2026); repository activity through September 2026.

Apple Developer Documentation: Playing Haptics; Icons. Human Interface Guidelines (accessed September 2026).

Tankala, S.: Skeleton Screens 101. Nielsen Norman Group, June 2023, reviewed September 2026.

Material Design 3: Design Tokens. [m3.material.io](https://m3.material.io) (accessed September 2026).

Lucide Icons ([lucide.dev](https://lucide.dev)) and Phosphor Icons ([phosphoricons.com](https://phosphoricons.com)): coverage and weight systems (accessed September 2026).

Mobbin: real-interface library, 1,400+ apps; UIguana: 7,700+ screenshots, 126+ flows; Kombai Gallery: 20,000+ curated examples; Banani: free iOS UI library (pattern sources).

Page Flows (flow capture), SaaSFrame (SaaS interface structure), [saasui.design](https://saasui.design) (dashboards).

Emerald UI ([shadcntemplates.com](https://shadcntemplates.com)); Awesome shadcn/ui resources ([shadcn.io](https://shadcn.io)); Awesome UI Libraries and Awesome Inspiration community indexes (GitHub).

Fuselab Creative: Mobile App Design Trends; Tubik Studio: 7 UI Design Trends of 2026; Pixelmatters: 7 UI design trends to watch in 2026; Envato: UX/UI design trends for 2026 (calm interfaces, AI-native transparency, accessibility-first).

Genre calibration: Hevy ([hevyapp.com](https://hevyapp.com), 2026 review coverage), MacroFactor (2026 independent reviews), Cal AI (2026 reviews, App Store and Play listings).

ui-ux-pro-max design-intelligence dataset: iconography, typography pairings, product palettes, UX guidelines, shadcn and React Native stack guidelines (bundled, accessed September 2026).

Nut AI Full QA Audit Report (companion volume, September 2026): interaction findings cross-referenced in Chapters 3, 10 and 11.

## B Current token dump (verbatim, for migration reference)

Table B.1 src/theme/tokens.ts values as audited

| Group          | Values                                                                                                                          |
|----------------|---------------------------------------------------------------------------------------------------------------------------------|
| Ink ramp       | 900 #0B0B0F, 800 #16161C, 700 #22222B, 600 #3A3A46, 500 #5C5C6B, 400 #8A8A99, 300 #B8B8C4, 200 #DCDCE4, 100 #EFEFF3, 50 #F7F7FA |
| Macro identity | protein #3E7BFA, carbs #F2A93B, fat #7B5EA7 (dark: #6E9BFF, #F5BC63, #A288CC)                                                   |
| Semantic       | uncertain #8B7BD8 on #F1EEFB; safety #D5453B on #FDEDEC; affirm #2E9E6B (dark: #A99AE6/#241F38, #F0655B/#3A1E1C, #4FBE8C)       |
| Space          | xs 4, sm 8, md 12, lg 16, xl 24, xxl 32, xxxl 48                                                                                |
| Radius         | sm 8, md 12, lg 16, xl 24, pill 999                                                                                             |
| Type           | hero 56/800, title 28/700, heading 20/600, body 16/400, bodyStrong 16/600, label 14/500, caption 13/400, micro 11/500           |
| Motion         | instant 90, fast 160, base 240, slow 380, reveal 520 (x motionScale)                                                            |
| Target         | MIN_TAP_TARGET 44 (lint-enforced)                                                                                               |
| Missing        | no elevation/shadow tokens, no accent slot, no state-layer tokens, dark contrast unverified                                     |

## C Screen inventory and transformation tag

**Table C.1 All 52 screens, grouped, with blueprint reference**

| Screen group (count)                                                                | Transformation                      |
|-------------------------------------------------------------------------------------|-------------------------------------|
| Onboarding (22 files)                                                               | Rebuild to 12 (8.1)                 |
| Home tab                                                                            | Instrument pass (8.2)               |
| Food tab                                                                            | Write-surface rebuild (7.1, 8.3)    |
| Train tab                                                                           | Hevy pattern (8.5)                  |
| Progress tab                                                                        | Hierarchy pass (8.6)                |
| Profile tab                                                                         | Grouped list + sub-pages (8.8)      |
| camera / result / checkin                                                           | Confidence + sheets (8.4)           |
| food-search / food-review / meal-detail                                             | Flow decomposition (8.3)            |
| saved-foods / recipes / dish-composer / indian-dishes / custom-food                 | Primitives + dead-control fix (8.3) |
| workout / log-exercise / search / exercise-detail / routines / programs / equipment | Context menus + set table (8.5)     |
| assistant / weekly-report / monthly-report / ReportScreen                           | Chat polish + card unity (8.6, 8.7) |
| provider-settings / edit-goals / data-methods / log-weight                          | Disclosure + diagnostics (8.8)      |

End of report. This document changes no application code; it is the specification the next implementation session should execute against, starting with Wave 1.